

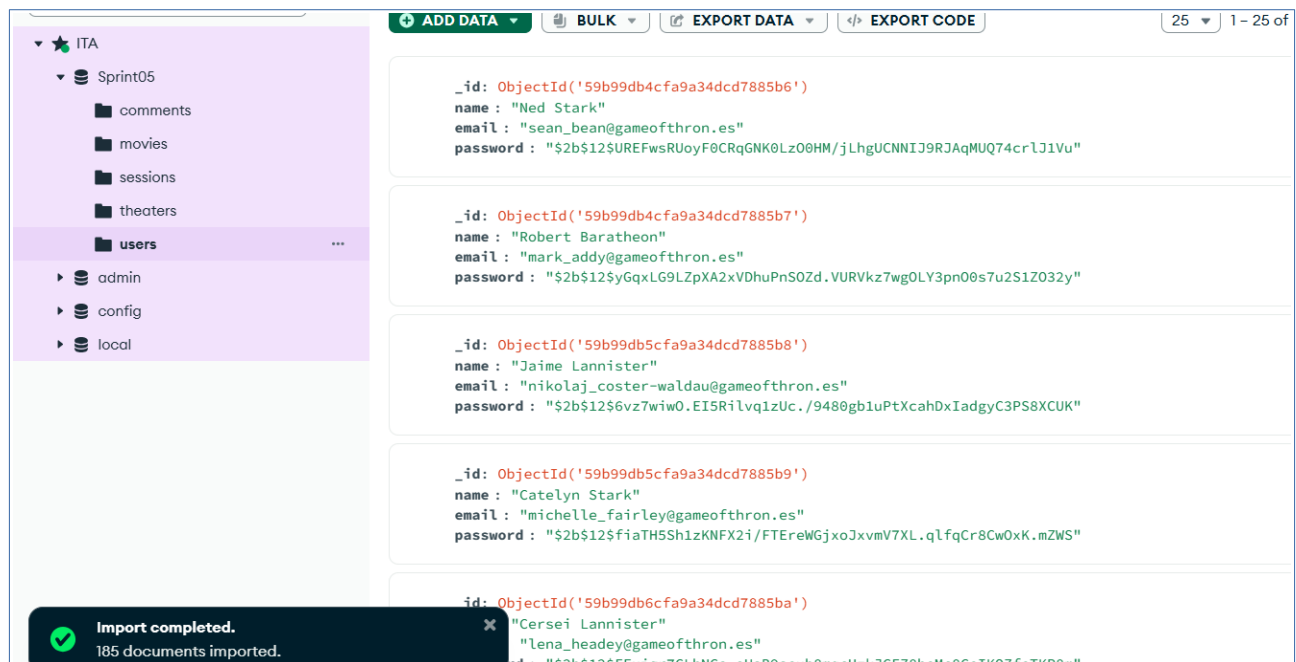
SPRINT 05

Tasca S5.01. Consultes amb MongoDB

Crea una base de dades amb MongoDB utilitzant com a col·leccions els arxius adjunts:

- comments.json
- movies.json
- sessions.json
- theaters.json
- users.json

All 5 .json files imported as collections (using Compass interface, ADD DATA):



- | | |
|-----------------|------------------|
| • comments.json | 50,304 documents |
| • movies.json | 23,539 documents |
| • sessions.json | 1 document |
| • theaters.json | 1,564 documents |
| • users | 185 documents |

EXERCISE 1

Mostra els 2 primers comentaris que hi ha en la base de dades.

There are 2 interface options:

- **Compass UI:** The **GUI** (Graphical User Interface)
- **mongosh:** The **CLI** (Command Line Interface)

Compass UI:

Type a query: { field: 'value' } or Explain Reset Find Options ▾

Project { field: 0 }

Sort { date: 1 } **Max Time MS** 60000

Collation { locale: Skip 0 **Limit** 2

Index Hint { field: -1 } or "indexName"

25 1 - 2 of 2

```

_id: ObjectId('5a9427648b0beeb695e67e')
name: "Mercedes Tyler"
email: "mercedes_tyler@fakegmail.com"
movie_id: ObjectId('573a1399f29313caabcec3ce')
text: "Optio totam dolores magni. Enim ratione fuga tempora voluptatum est cu..."
date: 1970-01-01T01:07:09.000+00:00

_id: ObjectId('5a9427658b0beeb6969efe')
name: "Jon Snow"
email: "kit_harington@gameofthron.es"
movie_id: ObjectId('573a13a4f29313caabd117df')
text: "Dolorem animi tempora ullam quas. Iusto nobis reprehenderit aspernatur..."
date: 1970-01-01T09:37:49.000+00:00

```

_mongosh:

```
db.comments.find({}).sort({date:1}).limit(2)
```

```

> db.comments.find({}).sort({date:1}).limit(2)
< {
  _id: ObjectId('5a9427648b0beeb695e67e'),
  name: 'Mercedes Tyler',
  email: 'mercedes_tyler@fakegmail.com',
  movie_id: ObjectId('573a1399f29313caabcec3ce'),
  text: 'Optio totam dolores magni. Enim ratione fuga tempora voluptatum est cumqu',
  date: 1970-01-01T01:07:09.000Z
}
{
  _id: ObjectId('5a9427658b0beeb6969efe'),
  name: 'Jon Snow',
  email: 'kit_harington@gameofthron.es',
  movie_id: ObjectId('573a13a4f29313caabd117df'),
  text: 'Dolorem animi tempora ullam quas. Iusto nobis reprehenderit aspernatur cu',
  date: 1970-01-01T09:37:49.000Z
}

```

Quants usuaris tenim registrats?

_mongosh:

```
db.users.countDocuments({})
// the empty {} denotes to include all (no selection/filter)
```

```

>_MONGOSH
> db.users.countDocuments({})
< 185

```

Compass:

ITA > Sprint05 > users Open MongoDB shell

Documents 185 Aggregations Schema Indexes 1 Validation

Type a query: { field: 'value' } or Explain Reset Find Options ▾

25 1 - 25 of 185

Quants cinemes hi ha en l'estat de Califòrnia?

Example of document fields in theaters:

```
_id: ObjectId('59a47286cfa9a3a73e51e72d')
theaterId: 1003
location: Object
  address: Object
    street1: "45235 Worth Ave."
    city: "California"
    state: "MD"
    zipcode: "20619"
  geo: Object
    type: "Point"
    coordinates: Array (2)
      0: -76.512016
      1: 38.29697
```

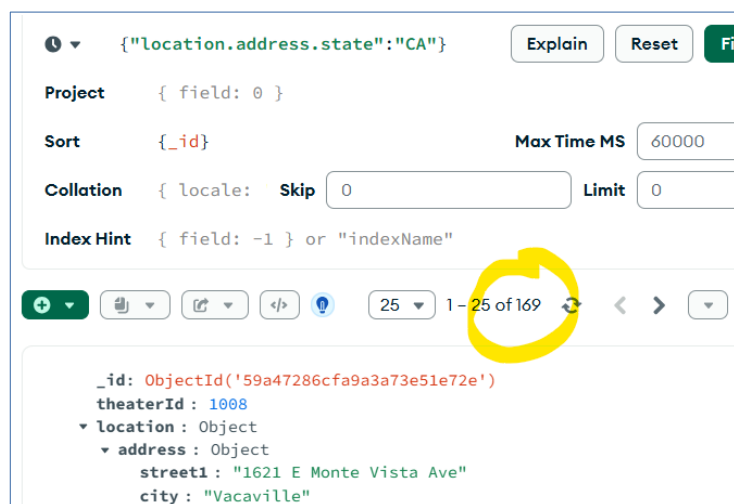
So, need to access location > address > state to filter the documents... which would be:

```
db.theaters.find({"location.address.state":"CA"})
// use the '.' path to find the level, and enclosed within "" so don't
get a syntax error!
```

_mongosh:

```
> db.theaters.countDocuments({"location.address.state":"CA"})
< 169
```

Compass:



Quin va ser el primer usuari/ària en registrar-se?

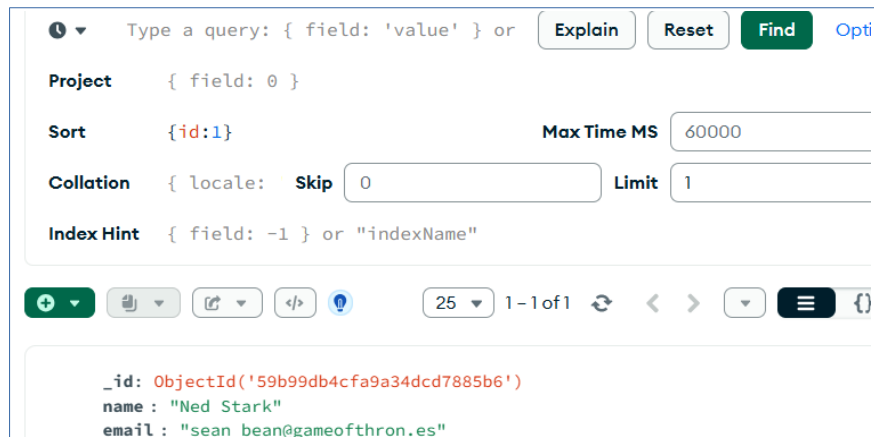
So, I need to .sort() the users (by _id) in ascending order (1) and with limit(1)

_mongosh:

```
db.users.find().sort({_id:1}).limit(1)
```

```
> _MONGOSH
> db.users.find().sort({_id:1}).limit(1)
< {
  _id: ObjectId('59b99db4cfa9a34dcd7885b6'),
  name: 'Ned Stark',
  email: 'sean_bean@gameofthron.es',
  password: '$2b$12$UREFwsRUoyF0CRqGNK0Lz00HM/jLhgUCNNIJ9RJAqMUQ74crLJ1Vu'
}
```

Compass:



The screenshot shows the MongoDB Compass interface. At the top, there's a query input field with the text "Type a query: { field: 'value' } or" and buttons for "Explain", "Reset", "Find", and "Opti". Below this, the query is displayed as `{ field: 0 }` under the "Project" section. The "Sort" section shows `{ id: 1 }` and a "Max Time MS" input field set to "60000". The "Collation" section shows `{ locale: Skip 0 }` and a "Limit" input field set to "1". The "Index Hint" section shows `{ field: -1 } or "indexName"`. Below the query fields, there's a toolbar with icons for adding, deleting, and saving queries, as well as a "25" dropdown and "1-1 of 1" text. The result is displayed in a JSON format: `{ "_id": ObjectId('59b99db4cfa9a34dcd7885b6'), "name": "Ned Stark", "email": "sean.bean@gameofthron.es" }`.

Quantes pel·lícules de comèdia hi ha en la nostra base de dades?

Example of movies document fields and levels:

```
_id: ObjectId('573a1390f29313caabcd4135')
plot: "Three men hammer on an anvil and pass a bottle of beer around."
genres: Array (1)
runtime: 1
cast: Array (2)
num_mflix_comments: 1
title: "Blacksmith Scene"
fullplot: "A stationary camera looks at a large anvil with a blacksmith behind it..."
countries: Array (1)
released: 1893-05-09T00:00:00.000+00:00
directors: Array (1)
rated: "UNRATED"
awards: Object
  wins: 1
  nominations: 0
  text: "1 win."
lastupdated: "2015-08-26 00:03:50.133000000"
year: 1893
imdb: Object
  rating: 6.2
  votes: 1189
  id: 5
  type: "movie"
tomatoes: Object
  viewer: Object
    rating: 3
    numReviews: 184
    meter: 32
  lastUpdated: 2015-06-28T18:34:09.000+00:00
```

... so I need to access the genres field, which is an array and could contain more than 1 value.

checking the values contained in the genres field, using `.distinct()`:

```
>_MONGOSH
> db.movies.distinct("genres")
< [
  'Action',      'Adventure', 'Animation',
  'Biography',   'Comedy',    'Crime',
  'Documentary', 'Drama',     'Family',
  'Fantasy',     'Film-Noir', 'History',
  'Horror',      'Music',     'Musical',
  'Mystery',     'News',      'Romance',
  'Sci-Fi',      'Short',     'Sport',
  'Talk-Show',   'Thriller',  'War',
  'Western'
]
```

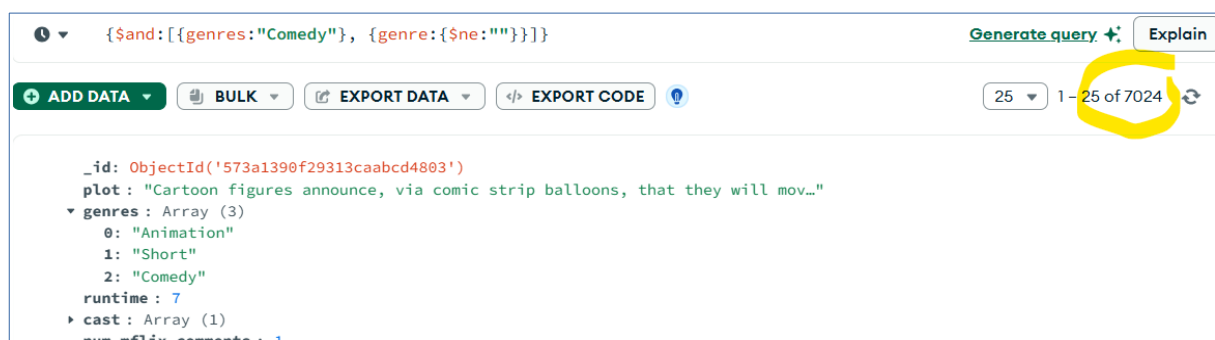
Then, use countDocuments() again to get the count for genre “Comedy”

_mongo:

```
db.movies.countDocuments({genres:"Comedy"})
```

```
>_MONGOSH
> db.movies.countDocuments({genres:"Comedy"})
< 7024
Sprint05> |
```

Compass: (added the ‘not null’ on genres, but not necessary, gives same result)



EXERCISE 2:

Mostra'm tots els documents de les pel·lícules produïdes en 1932, però que el gènere sigui drama o estiguin en francès.

Need to search using filters on year(1932) AND on genres(Drama) OR language (French)

First checking contents/accessing of the “languages” array:

```
> db.movies.distinct("languages")
< [
  'Ancient (to 1453)',
  'Old',
  'Abkhazian',
  'Aboriginal',
  'Acholi',
  'Afrikaans',
  'Aidoukrou',
  'Albanian'
```

_mongosh:

```
db.movies.find({$and: [{year:1932}, {$or: [{genres:"Drama"},
{languages:"French"}]}]})
```

...easier to read (use shift+enter to return without executing!):

```
db.movies.find({
  $and:[
    {year:1932},
    {$or:[
      {genres:"Drama"},
      {languages:"French"}
    ]}}
])
```

NB: although the AND is implicit in MongoDB, I have specified it in my code so it is easier to understand the exact filter that has been applied.

```
>_MONGOSH

> db.movies.find({$and:[{year:1932}, {$or:[{genres:"Drama"},{languages:"French"}]}]})
< {
  _id: ObjectId('573a1391f29313caabcd9458'),
  plot: 'A young artist draws a face at a canvas on his easel. Suddenly the mouth on the drawing
runtime: 55,
rated: 'UNRATED',
cast: [
  'Enrique Rivero',
  'Elizabeth Lee Miller',
  'Pauline Carton',
  'Odette Talazac'
],
num_mflix_comments: 1,
poster: 'https://m.media-amazon.com/images/M/MV5BYWY3ODE5ZWEtYjlmYi00NjA4LTk4ZWYtMzBhZDE5MjY0
title: 'The Blood of a Poet',
lastupdated: '2015-09-16 13:13:05.537000000',
languages: [
  'French'
],
released: 2010-05-20T00:00:00.000Z,
directors: [
  'Jean Cocteau'
],
writers: [
  'Jean Cocteau'
],
awards: {
  wins: 1,
  nominations: 0,
```

trying reducing the number of fields in the projection (fields shown) for easier checking of results:

```
db.movies.find({
  $and:[
    {year:1932},
    {$or:[
      {genres:"Drama"},
      {languages:"French"}
    ]}
  ]}, {title:1, year:1, genres:1, languages:1})
```

```
>_MONGOSH
> db.movies.find({
  $and:[
    {year:1932},
    {$or:[
      {genres:"Drama"},
      {languages:"French"}
    ]}
  ]}, {title:1, year:1, genres:1, languages:1})
< {
  _id: ObjectId('573a1391f29313caabcd9458'),
  title: 'The Blood of a Poet',
  languages: [
    'French'
  ],
  year: 1932
}
{
  _id: ObjectId('573a1392f29313caabcd99a3'),
  genres: [
    'Drama',
    'Fantasy',
    'Mystery'
  ],
  title: 'The Blue Light',
  languages: [
    'German',
    'Italian'
  ],
  year: 1932
}
```

Compass: (selecting the fields to show in the Project section...)

\$and:[{year:1932},{\$_or:[{genres:"Drama"},{languages:"French"}]}]}

Generate query ↗

Explain

Reset

Find

Project

{_id:0, title:1, year:1, genres:1, languages:1}

Sort

{ field: -1 } or [['field', -1]]

Max Time MS60000

Collation

{ locale: 'simple' }

Skip0Limit0

Index Hint

{ field: -1 } or "indexName"

EXPORT DATA

EXPORT CODE

251 – 18 of 18↺↻↷

title : "The Blood of a Poet"

▼ languages : Array (1)

0: "French"

year : 1932

▼ genres : Array (3)

0: "Drama"

1: "Fantasy"

2: "Mystery"

title : "The Blue Light"

▼ languages : Array (2)

0: "German"

1: "Italian"

year : 1932

EXERCISE 3:

Mostra'm tots els documents de pel·lícules estatunidenques que tinguin entre 5 i 9 premis que van ser produïdes entre 2012 i 2014.

So, need to find:

countries:"USA" AND {"awards.wins": {\$gte:5, \$lte:9}} AND {year: {\$gte:2012, \$lte:2014}}

```
db.movies.find({$and:[
    {countries:"USA"},
    {"awards.wins": {$gte:5, $lte:9}},
    {year: {$gte:2012, $lte:2014}}
]})
```



```
>_MONGOSH
> db.movies.find({$and:[
    {countries:"USA"},
    {"awards.wins": {$gte:5, $lte:9}},
    {year: {$gte:2012, $lte:2014}}
]})
< {
  _id: ObjectId('573a13acf29313caabd29366'),
  fullplot: "The manager of the negative assets sector of Life magazine, Walter Mitty, has been working for sixteen years for the ma
  imdb: {
    rating: 7.4,
    votes: 211230,
    id: 359950
  },
  year: 2013,
  plot: 'When his job along with that of his co-worker are threatened, Walter takes action in the real world embarking on a global j
  genres: [
    'Adventure',
    'Comedy',
    'Drama'
  ],
  rated: 'PG',
  metacritic: 54,
  title: 'The Secret Life of Walter Mitty',
  lastupdated: '2015-08-31 00:10:51.747000000',
  languages: [
    'English',
    'Spanish',
    'Icelandic'
  ],
}
```

... and, if preferred, reducing the columns shown:

```
db.movies.find({$and:[
    {countries:"USA"},
    {"awards.wins": {$gte:5, $lte:9}},
    {year: {$gte:2012, $lte:2014}}
]})
,{title:1, countries:1, "awards.wins":1, year:1})
```

```
>_MONGOSH
> db.movies.find({$and:[
    {countries:"USA"},
    {"awards.wins": {$gte:5, $lte:9}},
    {year: {$gte:2012, $lte:2014}}
]})
,{title:1, countries:1, "awards.wins":1, year:1})
< {
  _id: ObjectId('573a13acf29313caabd29366'),
  year: 2013,
  title: 'The Secret Life of Walter Mitty',
  awards: {
    wins: 6
  },
  countries: [
    'USA',
    'Canada'
  ]
}
{
  _id: ObjectId('573a13b5f29313caabd45772'),
  title: 'The Croods',
  awards: {
    wins: 8
  },
  year: 2013,
  countries: [
    'USA'
  ]
}
```

NIVELL 2

EXERCISE 1:

Compte quants comentaris escriu un usuari/ària que utilitza "GAMEOFTHRON.ES" com a domini de correu electrònic.

Need to use \$regex operator (similar to LIKE in SQL): `.find({fieldName:{$regex:"text"}})`

_mongosh:

```
db.comments.countDocuments({email:{$regex:"gameofthron.es"}})
```

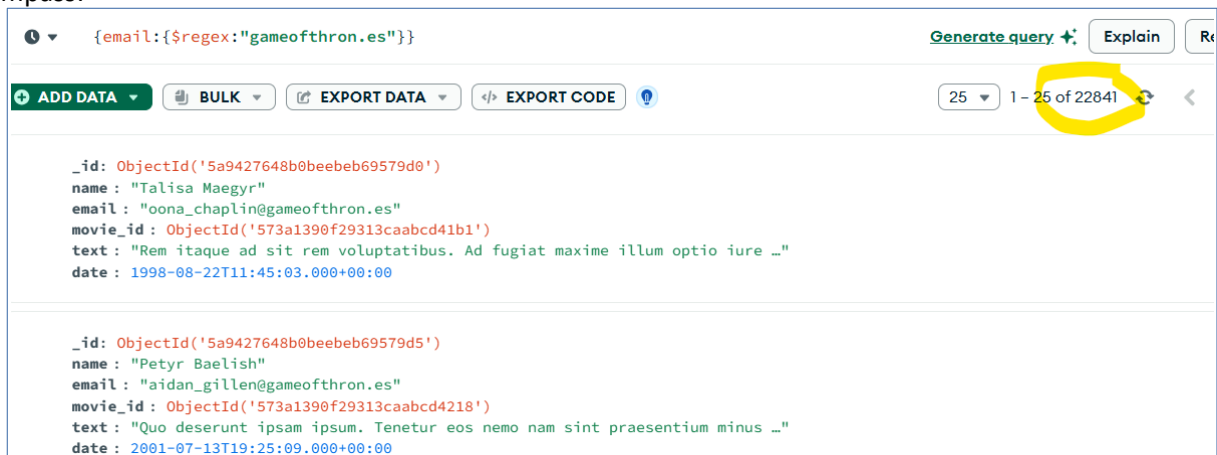
```
> db.comments.countDocuments({email:{$regex:"gameofthron.es"}})
< 22841
```

... and if the intention was to find "GAMEOFTHRON.ES" (specifically in uppercase, as MongoDB is case-sensitive):

```
db.comments.countDocuments({email:{$regex:"GAMEOFTHRON.ES"}})
```

```
> db.comments.countDocuments({email:{$regex:"GAMEOFTHRON.ES"}})
< 0
```

Compass:



EXERCISE 2:

Quants cinemes hi ha en cada codi postal situats dins de l'estat Washington D. C. (DC)?

Checking first selecting by state "DC": = 3 documents only

```
db.theaters.countDocuments({"location.address.state":"DC"})
```

```
> _MONGOSH
> db.theaters.countDocuments({"location.address.state":"DC"})
< 3
```

With following code I can see just 1 theater per postcode, but imagine it is expected we use \$group aggregation pipeline (see further below):

```
db.theaters.find({"location.address.state":"DC"},
{"location.address.state":1, "location.address.zipcode":1})
```

```
> db.theaters.find({"location.address.state":"DC"},{"location.address.state":1, "location.address.zipcode":1})
< {
  _id: ObjectId('59a47286cfa9a3a73e51e785'),
  location: {
    address: {
      state: 'DC',
      zipcode: '20010'
    }
  }
}
{
  _id: ObjectId('59a47287cfa9a3a73e51ec33'),
  location: {
    address: {
      state: 'DC',
      zipcode: '20016'
    }
  }
}
{
  _id: ObjectId('59a47287cfa9a3a73e51ecd8'),
  location: {
    address: {
      state: 'DC',
      zipcode: '20002'
    }
  }
}
```

Using **\$match** (to filter by state) as first point in the **.aggregate([...])** pipeline for **\$group** (that groups the data) and the aggregate operator **\$sum:1** (that counts the documents in each group:

```
db.theaters.aggregate([
  {$match: { "location.address.state": "DC" }},
  {$group: {
    _id: "$location.address.zipcode",
    theatres_this_zipcode: { $sum: 1 }
  }}
])
```

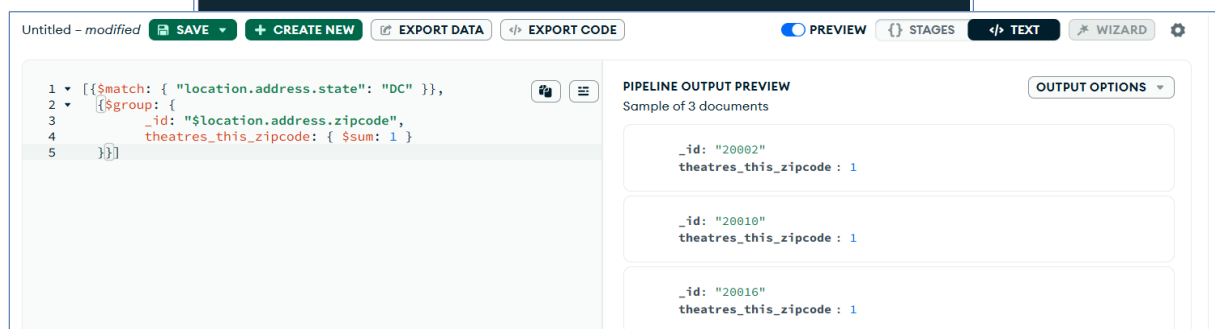
```

>_MONGOSH

> db.theaters.aggregate([
  {$match: { "location.address.state": "DC" }},
  {$group: {
    _id: "$location.address.zipcode",
    theatres_this_zipcode: { $sum: 1 }}
  }
])
< {
  _id: '20016',
  theatres_this_zipcode: 1
}
{
  _id: '20010',
  theatres_this_zipcode: 1
}
{
  _id: '20002',

```

Compass:



NIVELL 3

EXERCISE 1:

Troba totes les pel·lícules dirigides per John Landis amb una puntuació IMDb (Internet Movie Database) d'entre 7,5 i 8.

(before, without adding the {directors:{\$ne:""}}, it included all documents without this field)

```

db.movies.find(
  {$and: [
    {directors: "John Landis"},
    {directors:{$ne:""}},
    {"imdb.rating":{$gte:7.5}},
    {"imdb.rating":{$lte:8}}
  ]})

```

```

> db.movies.find(
  {$and:[
    {directors: "John Landis"},
    {directors:{$ne:""}},
    {"imdb.rating":{$gte:7.5}},
    {"imdb.rating":{$lte:8}}
  ]})
< {
  _id: ObjectId('573a1397f29313caabce6d94'),
  fullplot: "Faber College has one frat house so disreputable it will take anyone. It has a second
  imdb: {
    rating: 7.6,
    votes: 84834,
    id: 77975
  },
  year: 1978,
  plot: 'At a 1962 college, Dean Vernon Wormer is determined to expel the entire Delta Tau Chi Fra
  genres: [
    'Comedy'
  ],
  rated: 'R',
  metacritic: 82,
  title: 'Animal House',
  lastupdated: '2015-09-13 00:02:47.803000000',
  languages: [
    'English',
    'Italian'
  ],
  writers: [

```

Compass:

The screenshot shows the MongoDB Compass interface. At the top, a query is entered in the query editor: `{ $and: [ { directors: "John Landis" }, { directors: { $ne: "" } }, { "imdb.rating": { $gte: 7.5 } }, { "imdb.rating": { $lte: 8 } } ] }`. Below the query editor, the 'Project' tab is selected, showing the projection: `{ _id: 0, directors: 1, "imdb.rating": 1 }`. The 'Sort' tab is also visible, showing the sort order: `{ field: -1 } or [ [ 'field', -1 ] ]`. The 'Collation' tab shows the collation: `{ locale: 'simple' }`. The 'Index Hint' tab shows the index hint: `{ field: -1 } or "indexName"`. On the right side, there are buttons for 'Generate query', 'Explain', 'Reset', 'Find', and 'Options'. Below these buttons, there are input fields for 'Max Time MS' (60000), 'Skip' (0), and 'Limit' (0). At the bottom, there are buttons for 'EXPORT DATA' and 'EXPORT CODE'. The results section shows a list of documents. The first document is expanded, showing the 'imdb' object with 'rating: 7.6' and the 'directors' array with 'John Landis'. The second document is also expanded, showing the 'directors' array with 'John Landis' and the 'imdb' object with 'rating: 7.9'.

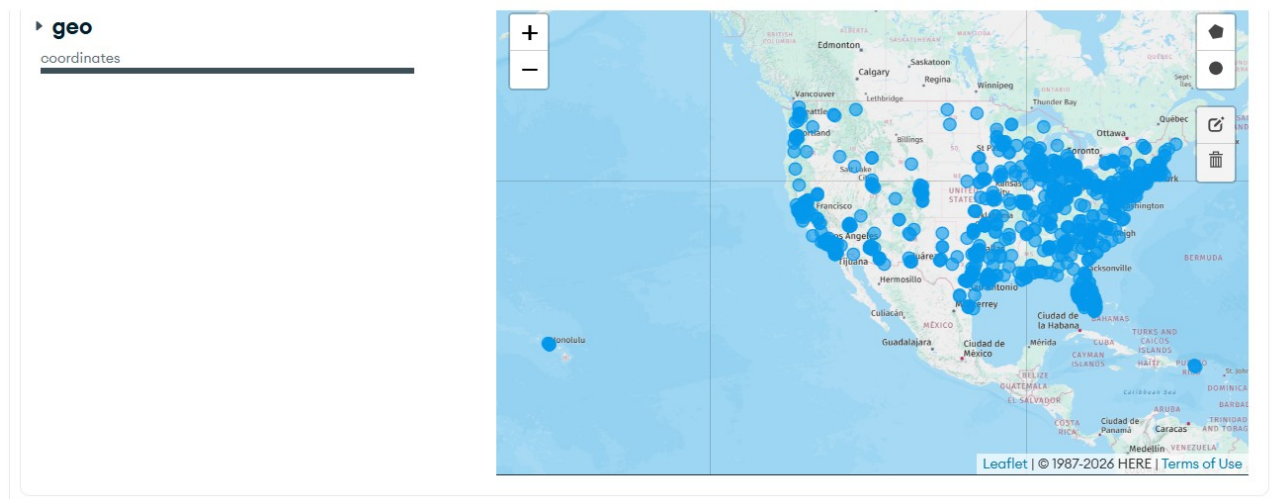
EXERCISE 2:

Mostra en un mapa la ubicació de tots els teatres de la base de dades.

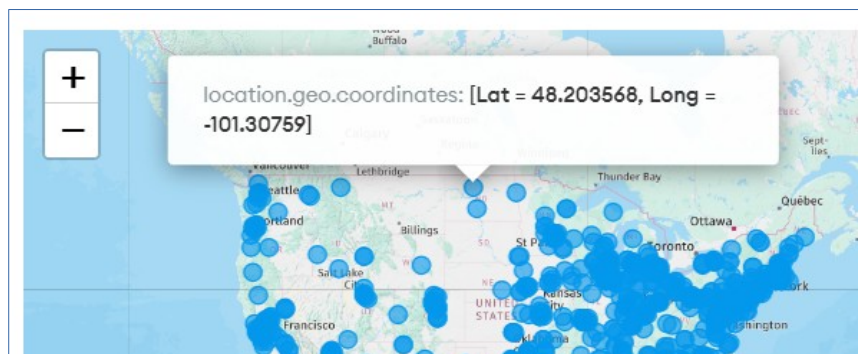
On the Schema tab of MongoDB Compass, the coordinates are plotted automatically on a map, but only a sample of 1000 locations:

EXPORT SCHEMA

This report is based on a sample of 1000 documents. [Learn more](#)



Also, can't select the information for each theater to be displayed:



To see all theaters plotted on a map, and choose the details to be displayed for each, I understand I can export my query to PowerBI to use the geographical maps tools of that software...

To do so I need to access this data (embedded object, containing the coordinates as an array) and take it to PowerBi...

```
"geo": {  
  "type": "Point",  
  "coordinates": [  
    -76.512016,  
    38.29697
```

```
  db.theaters.find(  
    {}, {  
      _id: false,  
      theaterId: true,  
      "location.address.street1": true,  
      "location.address.city": true,  
      "location.address.state": true,  
      "location.geo.coordinates": true}
```

)

```
>_MONGOSH
> db.theaters.find(
  {}, {
    _id:false,
    theaterId:true,
    "location.address.street1":true,
    "location.address.city":true,
    "location.address.state":true,
    "location.geo.coordinates.0":true,
    "location.geo.coordinates.1":true}
  )
< {
  theaterId: 1000,
  location: {
    address: {
      street1: '340 W Market',
      city: 'Bloomington',
      state: 'MN'
    },
    geo: {
      coordinates: []
    }
  }
}
```

... nearly, but showing coordinates as a list (array), I think I need them separate as longitude and latitude for the map visualisations in powerbi ...:

can use the **.aggregate** pipeline **\$project** , along with **\$arrayElemAt**, to 'unnest' them into separate named columns...

```
db.theaters.aggregate([
  {$project: {
    _id: 0,
    theaterId: 1,
    "location.address.street1": 1,
    "location.address.city": 1,
    "location.address.state": 1,
    longitude: { $arrayElemAt: ["$location.geo.coordinates", 0] },
    latitude: { $arrayElemAt: ["$location.geo.coordinates", 1] }
  }}
])
```

```

>_MONGOSH

> db.theaters.aggregate([
  {$project: {
    _id: 0,
    theaterId: 1,
    "location.address.street1": 1,
    "location.address.city": 1,
    "location.address.state": 1,
    longitude: { $arrayElemAt: ["$location.geo.coordinates", 0] },
    latitude: { $arrayElemAt: ["$location.geo.coordinates", 1] }
  }}
])
< {
  theaterId: 1000,
  location: {
    address: {
      street1: '340 W Market',
      city: 'Bloomington',
      state: 'MN'
    }
  },
  longitude: -93.24565,
  latitude: 44.85466
}
{
  theaterId: 1003,

```

OPTION 1: first step: stored my results from above query to a variable (array):

```

var myData = db.theaters.aggregate([
  {$project: {
    _id: 0,
    theaterId: 1,
    "location.address.street1": 1,
    "location.address.city": 1,
    "location.address.state": 1,
    longitude: { $arrayElemAt: ["$location.geo.coordinates", 0] },
    latitude: { $arrayElemAt: ["$location.geo.coordinates", 1] }
  }}
]).toArray();

```

then: exporting it to my :C drive, setting the path:

"C:\Users\DELL\Desktop\CODE\IT ACADEMY\MONGODB\Sprint 05"


```

var myData = db.theaters.aggregate([
  {$project: {
    _id: 0,
    theaterId: 1,
    "location.address.street1": 1,
    "location.address.city": 1,
    "location.address.state": 1,
    longitude: { $arrayElemAt: ["$location.geo.coordinates", 0] },
    latitude: { $arrayElemAt: ["$location.geo.coordinates", 1] }
  }}
]).toArray();
> const fs = require('fs');
fs.writeFileSync('C:\\Users\\DELL\\Desktop\\CODE\\IT ACADEMY\\MONGODB\\Sprint 05', JSON.stringify(myData, null, 2));
Error: EISDIR: illegal operation on a directory, open 'C:\\Users\\DELL\\Desktop\\CODE\\IT ACADEMY\\MONGODB\\Sprint 05'

```

... won't let me, try another folder:
 "C:\\Users\\DELL\\Descargas"

now doesn't give an error, but no file appears in the descargas folder :/

```

> const fs = require('fs');
fs.writeFileSync('C:\\Users\\DELL\\Descargas', JSON.stringify(myData, null, 2));
Sprint05>

```

OPTION 2: trying using the Compass UI....

Copied my query code into the aggregations tool, and asked Compass to generate the aggregation following my code.

The screenshot shows the MongoDB Compass interface. The 'Aggregations' tab is active, displaying a single stage named '\$project'. The pipeline editor contains the following JSON:

```

{
  "$project": {
    "_id": 0,
    "theaterId": 1,
    "location.address.street1": 1,
    "location.address.city": 1,
    "location.address.state": 1,
    "longitude": {
      "$arrayElemAt": [
        "$location.geo.coordinates",
        0
      ]
    },
    "latitude": {
      "$arrayElemAt": [
        "$location.geo.coordinates",
        1
      ]
    }
  }
}

```

Below the editor, the 'Output preview after \$project stage (Sample of 10 documents)' is shown. It displays two sample documents:

```

{
  "theaterId": 1000,
  "location": {
    "longitude": -93.24565,
    "latitude": 44.85466
  }
}

{
  "theaterId": 1003,
  "location": {
    "longitude": -76.512016,
    "latitude": 38.29697
  }
}

```

The interface also includes buttons for 'Generate', 'Run', 'Export Data', and 'Export Code'.

Saved the aggregation, then used the EXPORT DATA option to save the results as a JSON file, to then open in PowerBi. No transformation needed of the data when importing into PowerBi (coordinates array already unpacked).

PowerBi:

Simple map visualisation, with theatres plotted, and showing theaterId, Longitude, Latitude and City for each when the cursor rolls-over.

